Question A

1.

concurrency	requests	rps	p50_ms	p95_ms	p99_ms	errors	replicas	versions
1	1484	371.0	2.4	3.2	3.7	1484	1	{'n/a': 1484}
2	2385	596.2	3.0	4.0	11.7	2385	1	{'n/a': 2385}
4	3572	893.0	3.2	5.0	16.8	3572	1	{'n/a': 3572}
8	4509	1127.2	5.0	11.2	38.7	4509	1	{'n/a': 4509}
16	4765	1191.2	9.8	23.0	128.2	4765	1	{'n/a': 4765}

2. The saturation is around concurrency 8. The throughput gain per doubling of concurrency shrinks by a lot initially but at 8->16 it doesn't go down by that much. Past c=8, extra concurrency just adds queueing and p99 just gets way higher for not that much improvement in throughput.

   Capacity under the SLO is about 1,190 rps. Every tested level meets p95 ≤ 80 ms (the worst is 23.0 ms at c=16), so the SLO was never violated in this range. The limit is therefore set by saturation, not by the SLO.

3.

```
(myenv) vboxuser@Ubuntu:~/Downloads/Assign5/AiOps5$ python load_test.py --url https://lo
calhost:8000/v1/models/eta:predict --data data/eta_sample.csv --concurrency 1 --duration
 4 --batch 1
 concurrency  requests   rps  p50_ms  p95_ms  p99_ms  errors  replicas      versions
           1      1483 370.8     2.4     3.0     3.6    1483         1 {'n/a': 1483}
(myenv) vboxuser@Ubuntu:~/Downloads/Assign5/AiOps5$ python load_test.py --url https://lo
calhost:8000/v1/models/eta:predict --data data/eta_sample.csv --concurrency 1 --duration
 4 --batch 100
 concurrency  requests   rps  p50_ms  p95_ms  p99_ms  errors  replicas      versions
           1      1537 384.2     2.4     2.8     3.1    1537         1 {'n/a': 1537}
```

   Although the per-request latency barely changed, batching is about 100 times cheaper per row. This is because many rows can be processed at once in the batch cases.

4. I changed the number of cores from 1 to 2 because I thought that would cause the rps to double, but it did not change. Later I realized that that was because the server still only uses one CPU and just assigning it another one doesn't make it use it.

concurrency	requests	rps	p50_ms	p95_ms	p99_ms	errors	replicas	versions
1	1474	368.5	2.5	3.0	3.6	1474	1	{'n/a': 1474}
2	2777	694.2	2.6	3.1	9.2	2777	1	{'n/a': 2777}
4	3815	953.8	3.1	4.5	12.9	3815	1	{'n/a': 3815}
8	4297	1074.2	4.9	13.7	42.2	4297	1	{'n/a': 4297}
16	4456	1114.0	10.0	23.6	195.6	4456	1	{'n/a': 4456}

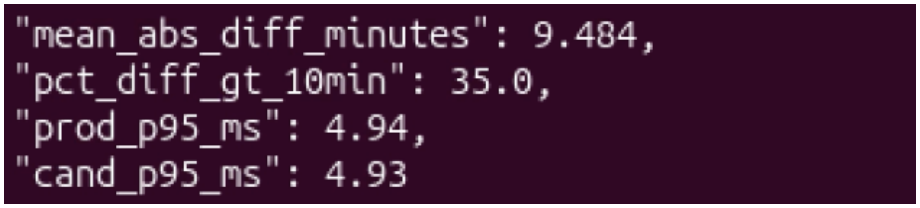
5. The mean usually doesn't show the slowest tail because most of the requests would be fast and they pull down the average. This would be more visible p95/p99 metrics.

Question B

1.

Version	Alias	Run
1	prod	eta-v1
2	candidate	eta-v2

2. Gate outputs are shown in the video

3. 

Question C

1. The inbuilt hash function randomizes the string hashing per process, so the same customer_id can hash to a different value in each process and in each restart. In Ray, the router can run in multiple replicas or be restarted or redeployed, which could cause inconsistencies in the customer_id hash if we used the inbuilt version.
2. Two reasons stickiness matters

- Product experience: If a customer's ETA came from v1 on one request and v2 on the next, the displayed ETA could jump around for the same trip. This looks buggy and the product no longer seems trustworthy anymore.
- Experiment validity: If customers flip between arms, each customer's outcomes mix both models, so the arms are contaminated and you can't attribute a difference in metrics to either model. Sticky assignment keeps the units of analysis cleanly separated, and it also makes ramping the weight safe, since customers already on the candidate stay there.

3. I set target_ongoing_requests=5. This is because each prediction takes only a few milliseconds and a replica can hold several requests in flight before the latency suffers. Scaling at 5 keeps replicas busy without adding one for every small burst. I used upscale_delay_s=10 so a brief spike doesn't trigger a new replica, which must reload the joblib bundle, but sustained load still scales out within about 10 s. I used a longer downscale_delay_s=30 so replicas aren't removed and re-created during short breaks. min_replicas=1 keeps it running so it never has to start from scratch, and max_replicas=3 is the most the 2-CPU budget allows.